Universidad Nacional de Ingeniería
Facultad de Ingeniería Industrial y de Sistemas

IMPACTA

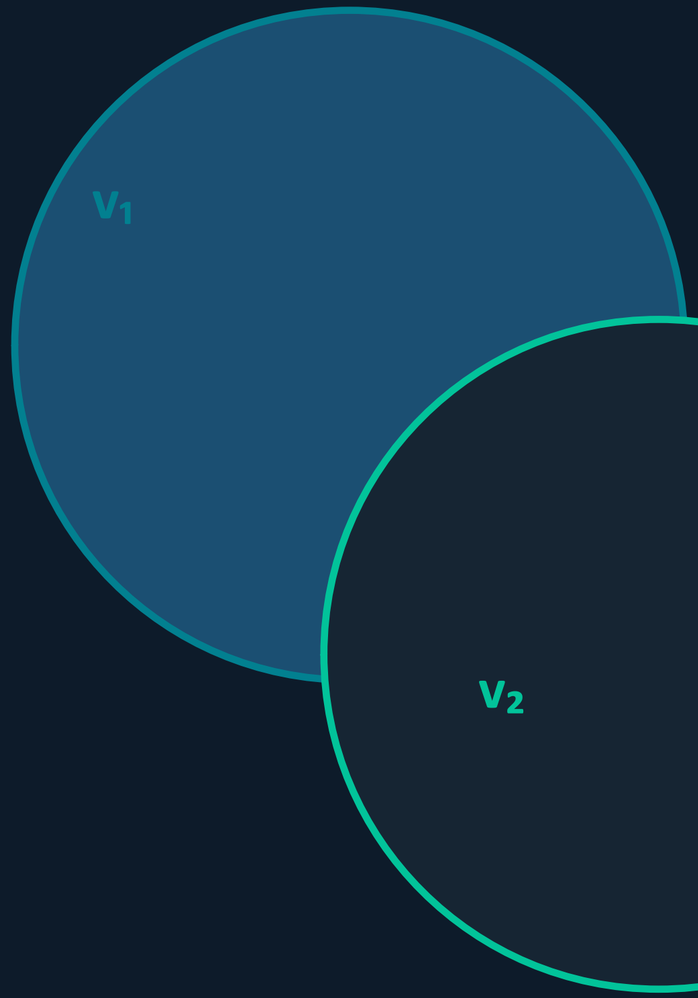
SIMULADOR DE PARTICULAS 2D

Lenguaje de programación 1 - SW303

Docente: Ing. Juan Luis Herencia Guerra

Integrantes:
Tom Jordan Pérez López
Raul Antonio Guzmán López
Ricardo Gaspar Reyes Campos
Sandro Giancarlo Chavez Vicaña

27 de abril, Lima 2026





Agenda de la Presentación

01

Tom Pérez

Requerimientos del sistema

03

Ricardo Reyes

Implementación en C

02

Sandro Sánchez

Ecuaciones Físicas

04

Raúl Guzmán

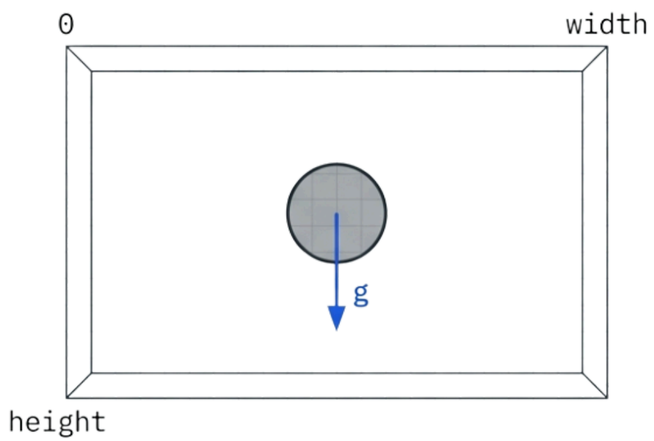
Diagramas y Arquitectura

01 Requerimientos del Sistema

Tom Jordan Pérez López

Perímetro y Restricciones del Sistema

Dominio Espacial



Entorno cerrado 2D. Paredes perfectamente rígidas.
Gravedad actuando únicamente en el eje Y (↓ positivo).

Parámetros Físicos

g (Gravedad):	Aceleración constante vertical (Default: 9.81 m/s^2)
μ (Fricción):	Coeficiente de amortiguamiento viscoso
e (Restitución):	Nivel de elasticidad en colisiones ($0 \leq e \leq 1$)

Restricciones Numéricas

dt: Diferencial de tiempo. Restricción crítica: **dt < 0.01s** para estabilidad.
steps: Iteraciones totales.
Colisiones: Estrictamente elásticas (conservación de momento y energía).

Tom Jordan

Requerimientos Funcionales del Sistema

RF1

Inicialización

N partículas con posiciones aleatorias, sin solapamiento inicial. Atributos: posición, velocidad, masa y radio configurables.

RF2

Integración de movimiento

Actualización de posición por tick mediante Euler Semi-Implícito con paso de tiempo dt configurable.

RF3

Gravedad

Aceleración gravitacional constante g (9.81 m/s^2) sobre eje vertical vy en cada tick.

RF4

Fricción viscosa

Coeficiente μ aplicado a vx y vy en cada tick, reduciendo gradualmente la energía cinética.

RF5

Detección de colisiones P-P

N·(N−1)/2 pares únicos evaluados por tick usando distancia euclidiana vs. suma de radios.

RF6

Resolución elástica 2D

Descomposición normal/tangencial, transferencia de momento y corrección de solapamiento.

RF7

Colisión con paredes

Reflexión con coeficiente e configurable (0=inelástico, 1=elástico perfecto).

RF8 Energía · RF9 CSV · RF10 Python · RF11 Benchmark

Requerimientos No Funcionales

RNF1	**Lenguaje y estándar**	C puro, C99/C11, compilado con gcc -Wall -Wextra -O2 -lm
RNF2	**Modularidad**	Mínimo 4 archivos: simulador.c, particula.h, fisicas.c, csv_export.c
RNF3	**Gestión de memoria**	malloc/calloc validados + free(). Apertura de archivos validada con fopen.
RNF4	**Rendimiento**	Soportar N=2 hasta N=1000 partículas. Usar flag -O2 + arreglos contiguos.
RNF5	**Portabilidad**	Compilable y ejecutable en Linux/macOS sin dependencias externas.

02 Ecuaciones Físicas

Sandro Giancarlo Sánchez Vicaña

El Motor de Integración (Euler Semi-Implícito)

Forward vs Semi-Implícito

Euler Tradicional:

$$x(t+dt) = x(t) + v(t) \cdot dt$$

Euler Semi-Implícito (Elegido):

$v(t+dt)$ calculado primero. Luego:

$$x(t+dt) = x(t) + v(t+dt) \cdot dt$$

Insight: Usar la velocidad actualizada para calcular posición mejora drásticamente la estabilidad numérica y la conservación de energía, evitando que el sistema explote a largo plazo.

Cronograma de Actualización

1. Aceleración

$$vx(t+dt) = vx(t) + ax(t) \cdot dt$$

2. Velocidad

$$vy(t+dt) = vy(t) + ay(t) \cdot dt$$

3. Posición

$$x(t+dt) = x(t) + vx(t+dt) \cdot dt$$

Dinámica de Fuerzas Continuas

Vector 1: Gravedad (Eje Y)

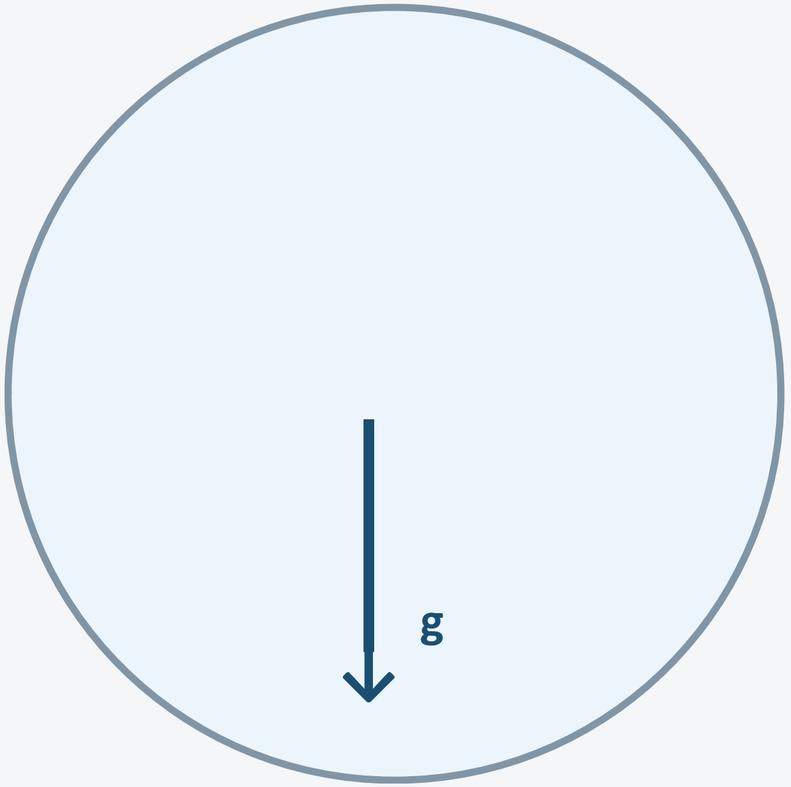
$$vy(t+dt) = vy(t) + g \cdot dt$$

Nota: Aceleración constante. ax = 0.

Vector 2: Fricción / Amortiguamiento Viscoso

Mecanismo: Decaimiento exponencial. Velocidad multiplicada por (1 - μ · dt).

⚠ **Invariante:** Se requiere μ < 1/dt para evitar inversión de velocidad.



Colisión con Paredes (Reflexión y Restitución)

Eje X — Paredes Izquierda / Derecha

Pared izquierda

Condición: x − radio ≤ 0

① Reposicionar: x = radio

② Reflexión: vx = |vx| · e

Pared derecha

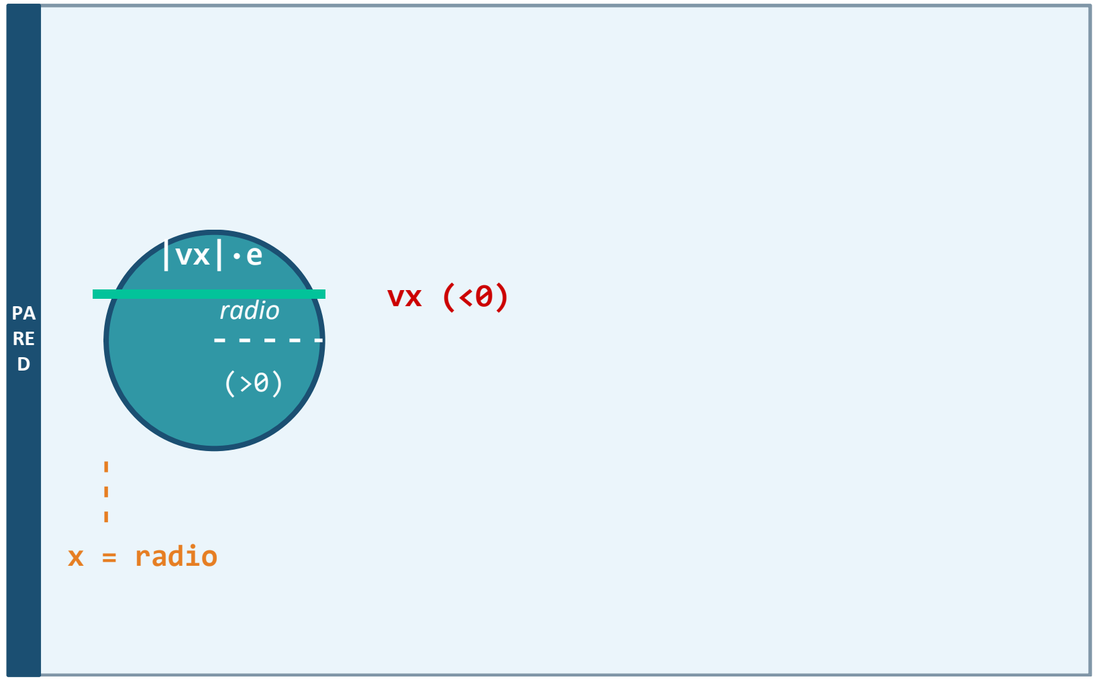
Condición: x + radio ≥ ancho

① Reposicionar: x = ancho − radio

② Reflexión: vx = −|vx| · e

Eje Y análogo: y−radio ≤ 0 → vy = |vy|·e | y+radio ≥ alto → vy = −|vy|·e

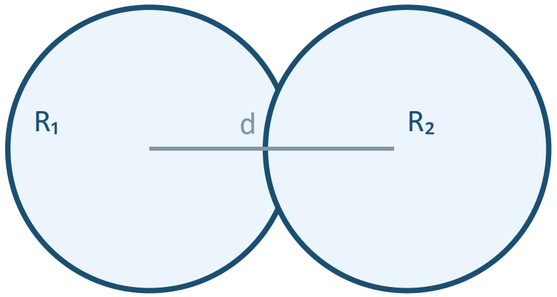
Diagrama — Rebote en Pared Izquierda



Coeficiente de restitución e (0 ≤ e ≤ 1)
e = 0 → colisión perfectamente inelástica (la partícula se detiene en la pared)
e = 1 → colisión perfectamente elástica (rebota sin pérdida de energía)

Resolución de Colisiones (Partícula-Partícula)

Fase 1: Detección



Condición: $d < (R_1 + R_2)$

$d = \sqrt{[(dx)^2 + (dy)^2]}$

Vector normal unitario:

$n = (dx/d, dy/d)$

Fase 2: Filtro Crítico

Velocidad relativa normal:

$vrel_n = \Delta v \cdot n$

Si $vrel_n \leq 0 \rightarrow$ ignorar

(partículas alejándose)

Fase 3: Impulso J

$J = 2 \cdot m_1 \cdot m_2 \cdot vrel_n / (m_1+m_2)$

$v_1_final = v_1 - (J/m_1) \cdot n$

$v_2_final = v_2 + (J/m_2) \cdot n$

(Acción / Reacción)

Paso 4: Corrección de solapamiento ponderada por masa — $factor_1 = m_2/(m_1+m_2)$

Auditoría de Estabilidad — Modelo Energético

Componentes de Energía

Cinética (E_k):

$$Ek = \Sigma \left[\tfrac{1}{2} \cdot m_i \cdot \text{sqrt}(vx_i^2 + vy_i^2) \right]$$

Potencial (E_p):

$$Ep = \Sigma \left[m_i \cdot g \cdot y_i \right]$$

(Invertida por convención Y-abajo)

$$Et = Ek + Ep \quad \text{(Energía Mecánica Total)}$$

Dependiente de masa y magnitud de velocidad.

El Principio de Conservación

Si $\mu = 0$ (sin fricción) y **$e = 1$** (perfectamente elástico), la Energía Mecánica Total (Et) debe formar una línea plana en el tiempo.

Cualquier fluctuación en energia.csv bajo estos parámetros corresponde exclusivamente al ruido de truncamiento de punto flotante (hardware) — validando la estabilidad del integrador Euler.

03 Implementación en C

Ricardo Gaspar Reyes Campos

Diseño de la Estructura de Datos (Struct)

```c
// particula.h

typedef struct {

    int    id;     // CSV traceability

    double x, y;   // Posición 2D

    double vx, vy; // Velocidad 2D

    double masa;   // Propiedad inercial

    double radio;  // Propiedad espacial

} Particula;
```

Anatomía de Memoria

`int id`

Traceability para CSV — agrupa filas por partícula en Python

`double x, y`

Centroide geométrico. Actualizado en Euler: x(t+dt) = x(t) + vx(t+dt)·dt

`double vx, vy`

Variables más volátiles. Modificadas por gravedad, fricción e impulsos

`double masa`

Inercia: determina factor de solapamiento y divisor de impulso J

`double radio`

Límites euclidianos de colisión y punto de rebote en paredes

double sobre float: mitiga error de truncamiento en miles de ticks

Avance de Código — Pipeline del Tick

```
// FASE 1: Integración
double factor_friccion = 1.0 - (mu * dt);  // Pre-calculado fuera del bucle
for (int i = 0; i < N; i++) {
    sistema[i].vy += g * dt;       // 1.1 Gravedad
    sistema[i].vx *= factor_friccion; sistema[i].vy *= factor_friccion;  // 1.2 Fricción
    sistema[i].x  += sistema[i].vx * dt;  // 1.3 Posición (vel. actualizada → Euler Semi-Implícito)
    sistema[i].y  += sistema[i].vy * dt;
}
// FASE 2: Colisión con Paredes
if (sistema[i].x - sistema[i].radio <= 0) {
    sistema[i].x = sistema[i].radio;
    sistema[i].vx = fabs(sistema[i].vx) * e;  // Reflexión + restitución
}
// FASE 3 & 4: P-P + Energía/CSV
resolver_colisiones_particulas(sistema, N);  calcular_y_exportar(sistema, N, tick);
```

Análisis Algorítmico y Rendimiento (Big-O)

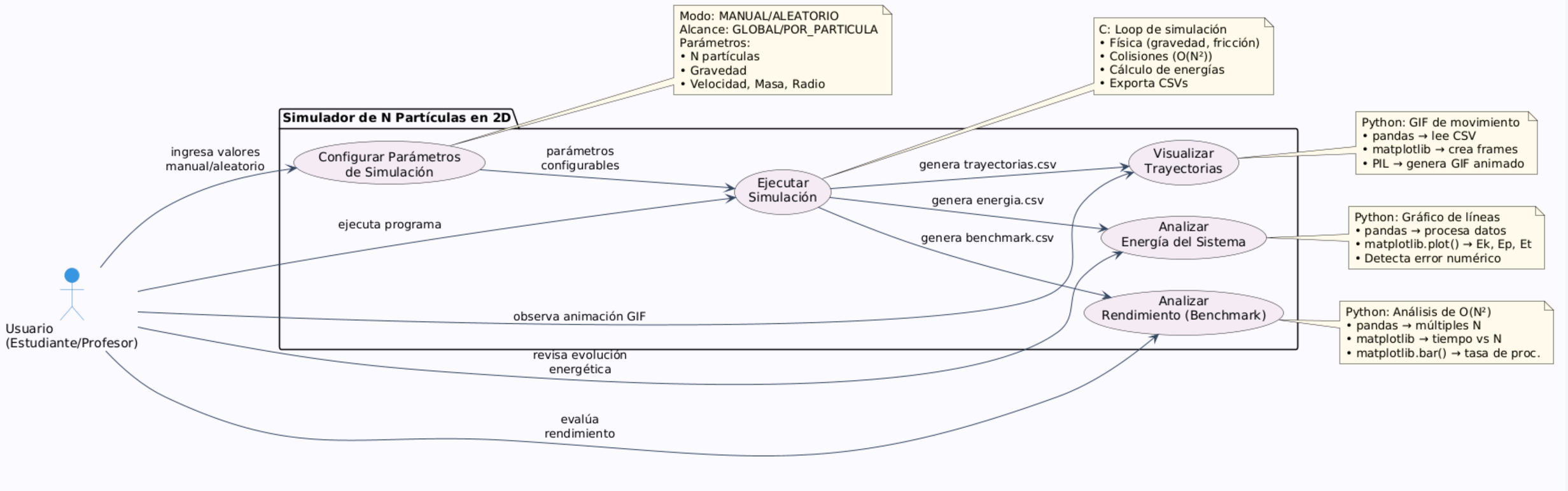
1	**Fase 1 — Integración** 5 operaciones escalares por partícula	$O(5N) \rightarrow O(N)$
2	**Fase 2 — Muros** 4 comparaciones espaciales por partícula	$O(4N) \rightarrow O(N)$
3	**Fase 4 — Energía/CSV** Sumatorias lineales	$O(2N) \rightarrow O(N)$
4	**Fase 3 — Colisiones P-P** Evaluación cruzada iterativa — CUELLO DE BOTELLA	$O(N(N-1)/2) \rightarrow O(N^2)$

Síntesis: $O(11N + (N^2-N)/2) \approx \mathbf{O(N^2)}$ por tick → Total: **$O(Ticks \cdot N^2)$** — justifica flag -O2 y arreglos contiguos en caché

04 Diagramas y Arquitectura

Raúl Antonio Guzmán López

Diagrama de caso de uso



Diagramas de estructuras

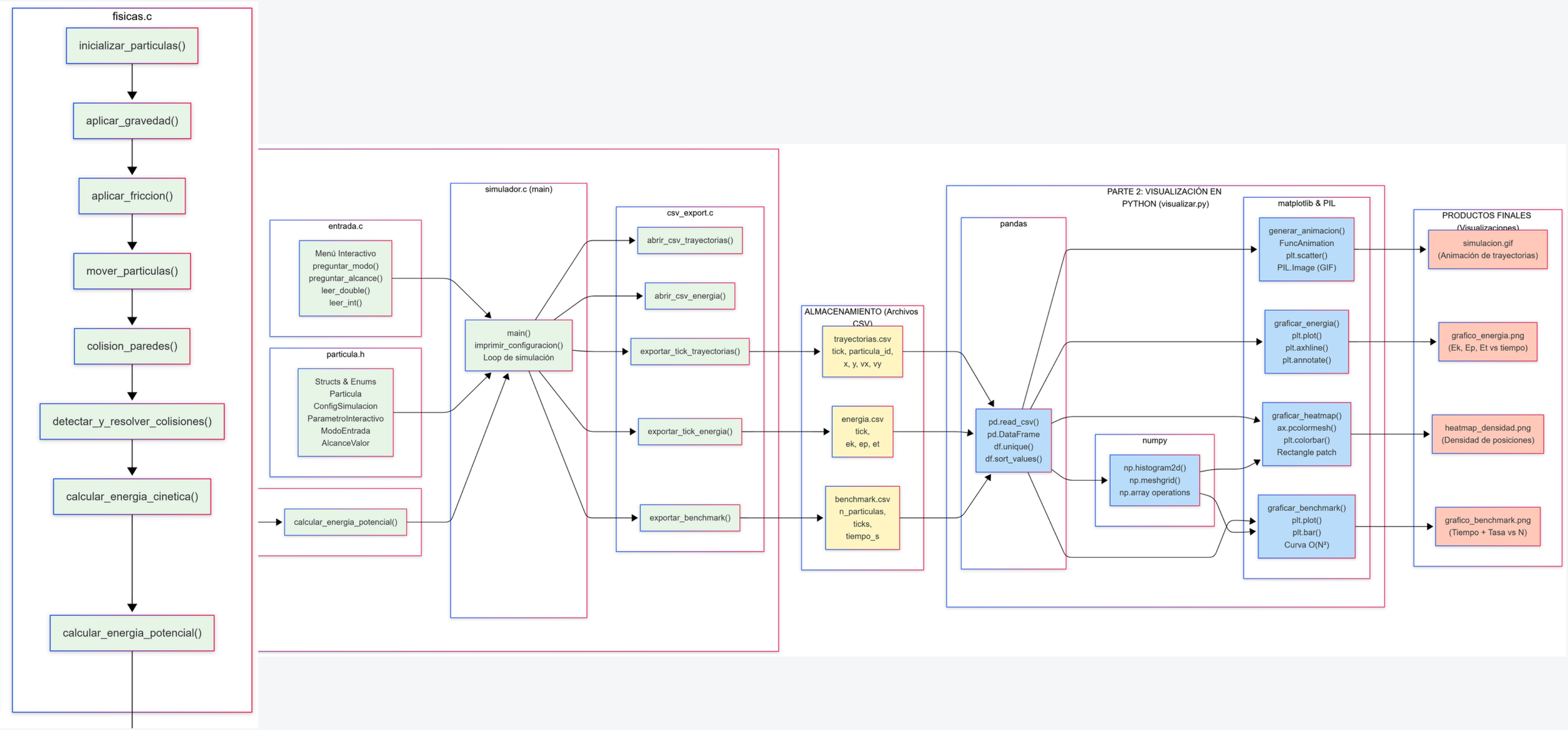


Diagrama de secuencia

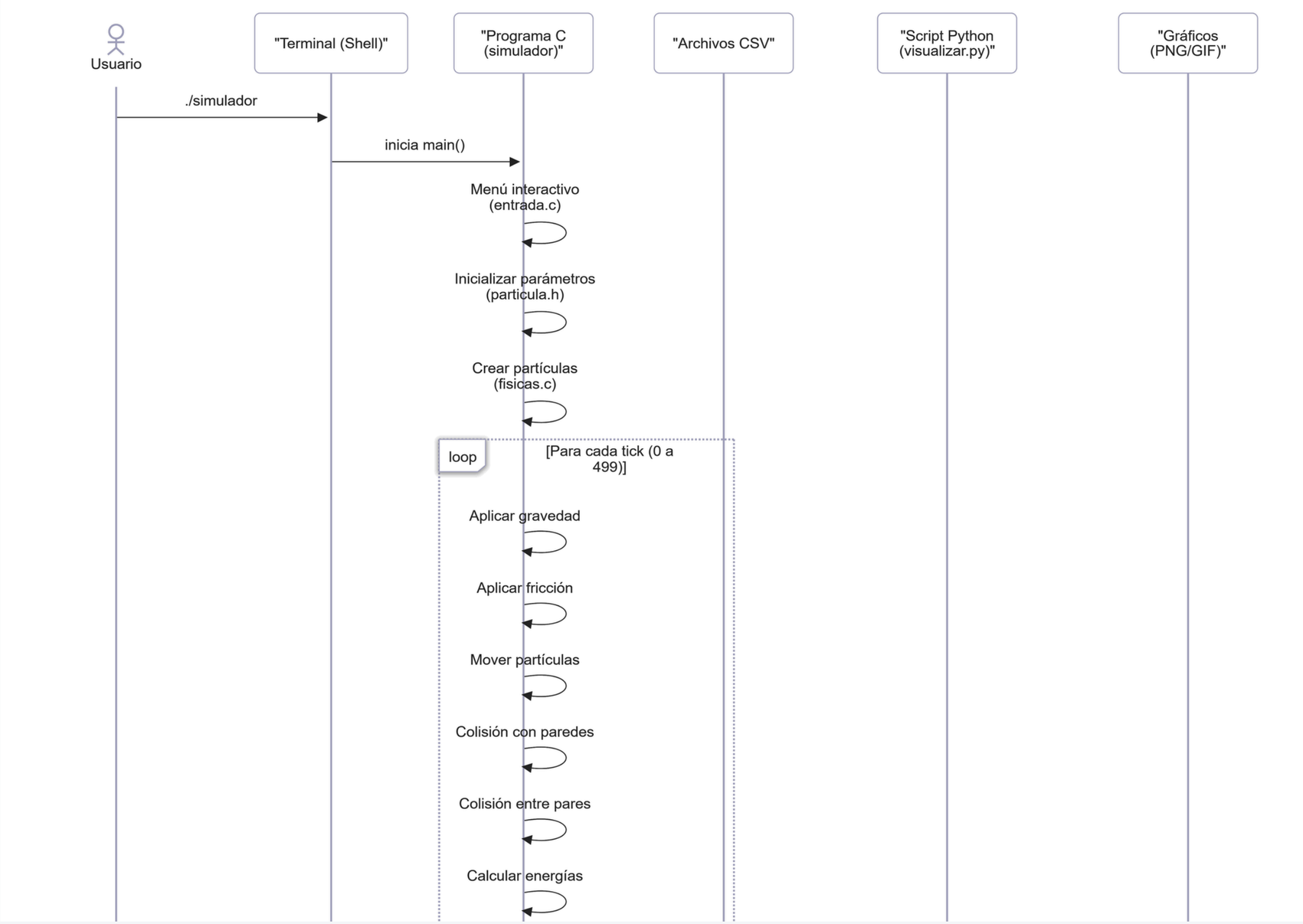
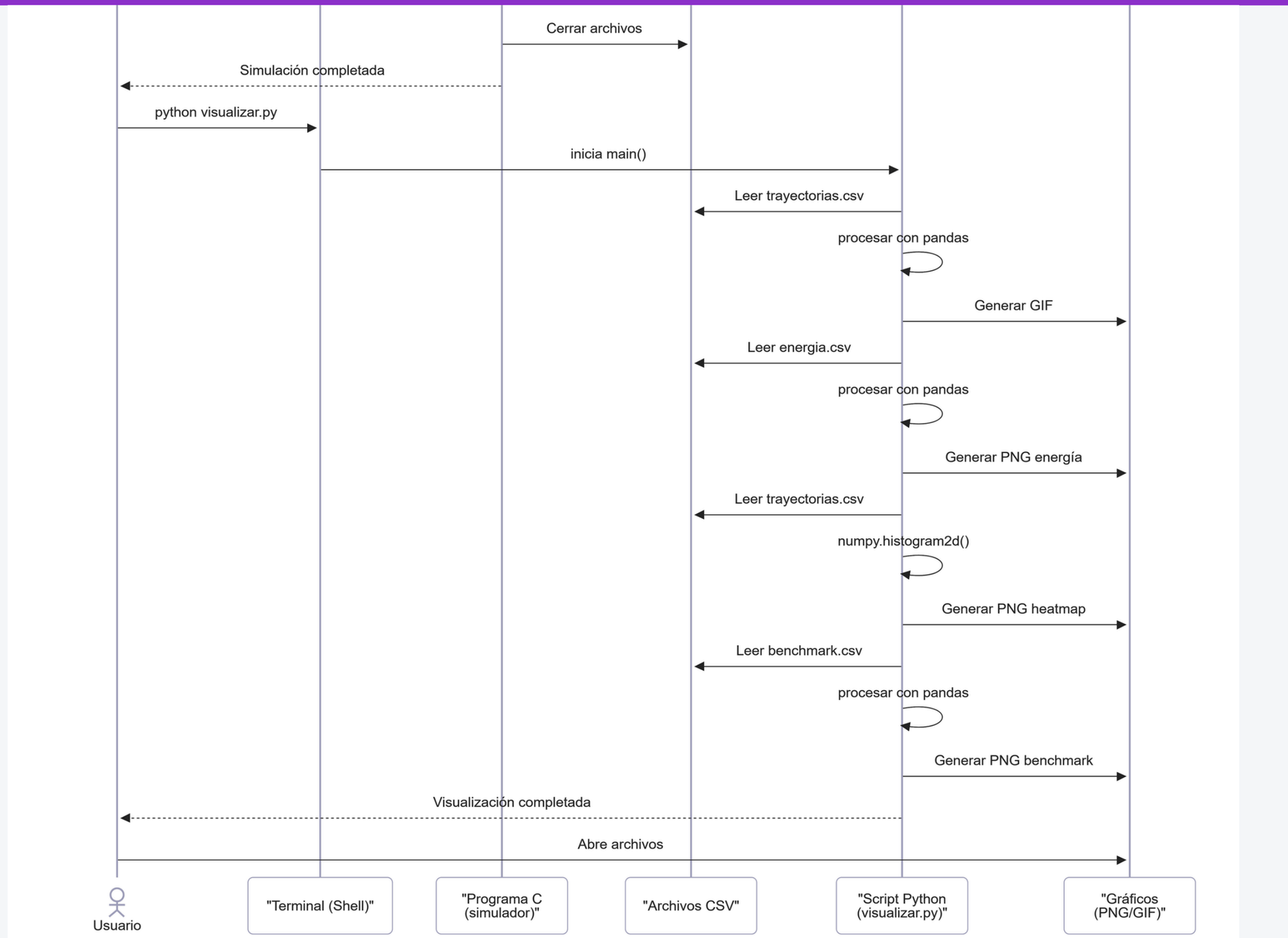


Diagrama de secuencia



¡Muchas gracias!